Royal Paradise Luxury Hotel



Table of Contents

1. Project Overview 1
1.1 Assignment 2 summary 1
1.2 Website pages included 1
2. Website Structure 2
2.1 Royal Paradise Luxury Hotel Website Workflow 2
2.2 File structure, links and references 2
2.3 Representative navigation code 3
3. HTML Implementation Evidence 3
3.1 HTML5 media implementation 3
3.2 Semantic image, figure and caption 3
3.3 Room table and direct booking links 4
3.4 New HTML elements added for Assignment 2 4
4. JavaScript Implementation 5
4.1 Timer-based dynamic content and style 5
4.2 User-action DOM manipulation and event listeners 5
4.3 Custom input-validity checking 6
4.4 Form validation and prevented submission 6
4.5 Advanced JavaScript: live booking estimate 6
5. CSS Implementation 7
5.1 External stylesheet and design system 7
5.2 CSS 3D transforms and transitions 8
5.3 Grid, Flexbox and responsive design 8
5.4 Form validation states and UI feedback 9
6. Requirement Traceability 9
7. Screenshots 11
7.1 Home Page 11
7.2 Rooms Page 12
7.3 Booking Page 12
7.4 Gallery Page 13
7.5 Contact Page 13
8. Conclusion 13

1. Project Overview

The Royal Paradise Luxury Hotel has a total of five pages of online activity. The website was made with HTML5 and CSS3 and JavaScript and is for Assignment II. Building on what was done in the first assignment, HTML5 media, advanced CSS effects, responsive layouts, and interactive JavaScript are implemented in the project. The final simplest version of the project includes updates of limited time offers, the handling of events in the DOM, password verification and form validation at submission, and a calculation of the price of the booking in a quick and engaging way.

1.1 Assignment 2 summary

The final submission keeps the implementation focused on the required learning outcomes. It uses five HTML pages, one shared stylesheet and two external JavaScript files: main.js and booking.js.

Enhancement Area	Current Implementation	Main File(s)
HTML5 media	MP4 hotel video with browser controls plus JavaScript Play/Pause buttons	index.html, js/main.js
CSS Advanced	3D room flip cards and transition effects	rooms.html, css/style.css
Timer JavaScript	Offer text and CSS class change every 3 seconds	js/main.js
DOM / user action	Show/Hide Facilities button using DOM and addEventListener()	js/main.js
Input validity	Custom password rules: 8+ characters, uppercase, lowercase and number	js/booking.js
Form validation	Name, email, password, room and date checks; invalid submission prevented	booking.html, js/booking.js
Advanced JavaScript	Live room-price estimate using room data, date arithmetic and DOM updates	js/booking.js
Responsive UI	Grid/Flexbox layouts and media queries for smaller screens	css/style.css

1.2 Website pages included

The current project contains five user-facing HTML pages. All pages share the same header, navigation style and footer, while each page demonstrates a different part of the Assignment 2 implementation.

Page	File	Assignment 2 Functionality
Home	index.html	Hero section, timed offers, Show/Hide Facilities, semantic figure and HTML5 video with JavaScript controls

Page	File	Assignment 2 Functionality
Rooms	rooms.html	Three CSS 3D flip cards, direct booking links and a room price summary table
Booking	booking.html	Custom password validation, multi-field form validation, date checking, facilities and live booking estimate
Gallery	gallery.html	Responsive six-image gallery with captions and CSS hover/scale transitions
Contact	contact.html	Contact cards, mail/phone links and a booking call-to-action

2. Website Structure

The navigation model is intentionally simple. Every main page is available from the shared navigation bar. Room-card buttons open booking.html directly, and the Booking page performs validation and live price calculation in the browser. The Gallery page is a responsive image gallery without a JavaScript lightbox.

2.1 Royal Paradise Luxury Hotel Website Workflow

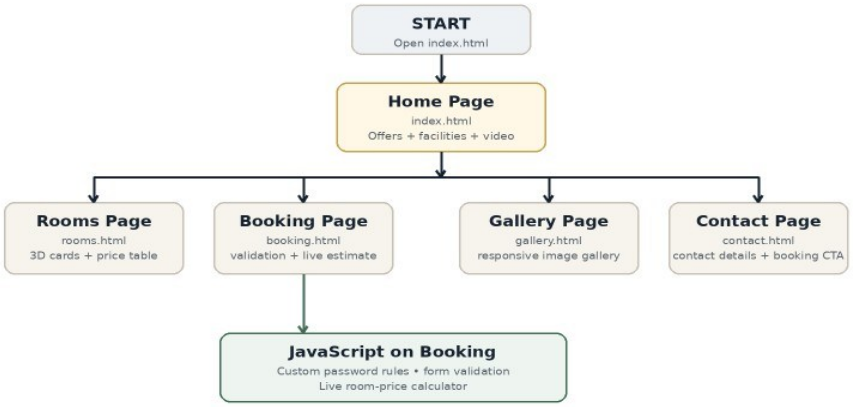


Figure 1. Current five-page website workflow.

2.2 File structure, links and references

The final submission separates HTML, CSS, JavaScript, images and media into clear folders. Relative paths are used for all internal pages and local assets.

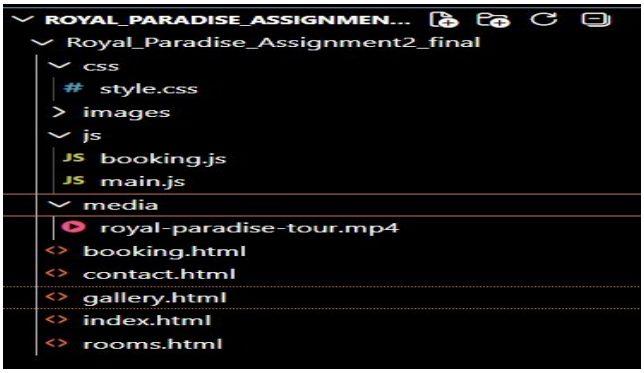


Figure 2. project file structure

- Same-directory page links use index.html, rooms.html, booking.html, gallery.html and contact.html.
- Room-card booking buttons use href="booking.html" with no URL query parameter.
- Child-directory references include css/style.css, js/main.js, js/booking.js, images/ and media/.
- The current project contains only two JavaScript files: main.js and booking.js.

2.3 Representative navigation code

NEW CODE HIGHLIGHT - Shared navigation

```
<nav class="main-nav" aria-label="Main navigation">
    <a href="index.html">Home</a>
    <a href="rooms.html">Rooms</a>
    <a href="booking.html">Booking</a>
    <a href="gallery.html">Gallery</a>
    <a href="contact.html">Contact</a>
</nav>
```

3. HTML Implementation Evidence

3.1 HTML5 media implementation

The Home page contains an HTML5 <video> element with an MP4 source and built-in browser controls. Two normal HTML buttons are also provided so main.js can call play() and pause() on the video. This demonstrates HTML5 media together with simple DOM event handling.

NEW CODE - HTML5 video and controls

```
<div class="video-controls">
    <button id="playVideo" class="button button-gold" type="button">Play Video</button>
    <button id="pauseVideo" class="button button-dark" type="button">Pause Video</button> </div>

<video id="hotelVideo" controls preload="metadata">
    <source src="media/royal-paradise-tour.mp4" type="video/mp4">
Your browser does not support HTML5 video.
</video>
```

3.2 Semantic image, figure and caption

The Home page uses semantic <figure> and <figcaption> elements for the swimming-pool image. The image also includes meaningful alternative text for accessibility.

NEW CODE - Semantic figure

```
<figure class="pool-figure">
    <img src="images/pool.jpg" alt="Swimming pool at Royal Paradise Hotel">
    <figcaption>Relax by our swimming pool</figcaption>
</figure>
```

3.3 Room table and direct booking links

The Rooms site comprises three CSS flip cards and a room price summary table with three columns. The links for booking are kept very straightforward: clicking on any of the room cards proceeds directly to booking.html. The actual selection of rooms is done on the Booking page.

NEW CODE - Direct room booking link

```
<a class="button button-light" href="booking.html">Book This Room</a>
```

NEW CODE - Room price table structure

```
<table class="room-table">
    <caption>Room Price Summary</caption>
    <thead>
        <tr><th>Room</th><th>Price</th><th>Main Feature</th></tr>
    </thead>
    <tbody>
        <tr><td>Luxury Suite</td><td>$500/night</td><td>Premium city view</td></tr>
        ...
    </tbody>
</table>
```

3.4 New HTML elements added for Assignment 2

New / Expanded Element	Use	Example Page
<video> and <source>	HTML5 hotel promotional media	Home
<button>	Show/Hide Facilities and Play/Pause video controls	Home
<article>	Facility cards, room cards and contact cards	Home / Rooms / Contact
<aside>	Live booking estimate panel	Booking
<fieldset> and <legend>	Grouped extra-facility options	Booking
<small>	Inline help and validation messages	Booking
<dl>, <dt>, <dd>	Structured live booking estimate	Booking
<figure> and <figcaption>	Semantic and gallery images with captions	Home / Gallery

4. JavaScript Implementation

Two external JavaScript files are used in the project. main.js handles Home-page interaction and video controls, while booking.js handles booking validation and the live estimate. This keeps the JavaScript easy to follow during an in-lab demonstration.

4.1 Timer-based dynamic content and style

The Home page changes the promotional offer every three seconds. An array stores both the message text and the CSS class. changeOffer() updates textContent and className, and setInterval() repeats the change. This satisfies Task 2(a).

NEW CODE - Timer-based live offer const
offers = [
{ text: "Welcome to Royal Paradise Luxury Hotel", style: "offer-gold" },
{ text: "Enjoy 10% off selected weekday stays", style: "offer-blue" },
{ text: "Relax by the pool and enjoy fine dining", style: "offer-green" }
];

let offerIndex = 0;

function changeOffer() { offerIndex = (offerIndex + 1) % offers.length;
offerElement.textContent = offers[offerIndex].text;
offerElement.className = `offer-message ${offers[offerIndex].style}`; }

if (offerElement) {
setInterval(changeOffer, 3000);

4.2 User-action DOM manipulation and event listeners

The Show Facilities button showcases DOM selection, click event listener, class toggling and update text feature of JavaScript. A click on this button results in adding or removing class hidden and changing the text of the button.

NEW CODE - Show/Hide Facilities
const facilityButton = document.getElementById("facilityButton"); const
facilityDetails = document.getElementById("facilityDetails");

if (facilityButton && facilityDetails) {
facilityButton.addEventListener("click", function () {
facilityDetails.classList.toggle("hidden");

if (facilityDetails.classList.contains("hidden")) {
facilityButton.textContent = "Show Facilities";
} else { facilityButton.textContent = "Hide
Facilities";
}
}); }

4.3 Custom input-validity checking

The booking password is checked using custom JavaScript. A valid password must contain at least eight characters, one uppercase letter, one lowercase letter and one number. The same validatePassword() function is used for live feedback and submit-time checking. This satisfies Task 2(c).

NEW CODE - Custom password rules

```
function validatePassword() {  const value = password.value;  const hasLength = value.length >= 8;  const hasUppercase = /[A-Z]/.test(value);  const hasLowercase = /[a-z]/.test(value);  const hasNumber = /[0-9]/.test(value);

  if (!hasLength || !hasUppercase || !hasLowercase || !hasNumber) {
return showError(  password,
    "passwordError",
    "Password needs 8+ characters, uppercase, lowercase and a number."
  );
  }

  return showValid(password, "passwordError"); }
```

4.4 Form validation and prevented submission

The submit listener checks five validation groups: full name, email, password, room selection and dates. The date function separately checks both check-in and check-out. Invalid submissions are prevented with event.preventDefault(). Because the assignment site has no server, a valid form is also kept on the same page and a success message is displayed. This satisfies Task 2(d).

NEW CODE - Submit-time validation

```
bookingForm.addEventListener("submit", function (event) {
const nameOk = validateName();  const emailOk = validateEmail();  const passwordOk = validatePassword();
const roomOk = validateRoom();  const datesOk = validateDates();

  if (!nameOk || !emailOk || !passwordOk || !roomOk || !datesOk) {
event.preventDefault();  formMessage.textContent = "Please fix the highlighted fields before submitting.";  formMessage.className = "form-message error-text";
return;
  }

  event.preventDefault();  formMessage.textContent = "Form is valid. Your booking details are ready to submit.";  formMessage.className = "form-message success-text";
});
```

4.5 Advanced JavaScript: live booking estimate

updateEstimate() reads the selected room, calculates the number of nights from the two dates, collects checked facilities and updates the estimate panel. The room total is calculated as room rate × number

of nights. The function runs whenever the room, dates or facility checkboxes change. This is the main Task 2(e) feature.

NEW CODE - Live estimate excerpt

```
const room = roomData[roomType.value];
let nights = 0;

if (checkIn.value && checkOut.value) {
  const start = new Date(checkIn.value);
const end = new Date(checkOut.value);
const difference = end - start;

  if (difference > 0) {        nights = Math.ceil(difference /
(1000 * 60 * 60 * 24));
  }
}

const selectedFacilities = Array.from(                document.querySelectorAll(".facility-
checkbox:checked")
).map(function (item) {
return item.value;
});

estimateTotal.textContent = room && nights > 0
  ? `$${room.rate * nights}`
: "$0";
```

5. CSS Implementation

5.1 External stylesheet and design system

All five HTML pages reference css/style.css. The stylesheet defines a shared color palette, typography, buttons, cards, shadows, spacing and responsive layouts. CSS custom properties keep the visual identity consistent across the project.

NEW CODE - Current CSS custom properties

```
:root {
  --gold: #c9a24d;
  --dark: #162230;
  --blue: #315c7d;
  --green: #426d57;
  --light:  #f7f4ed;        --
white: #ffffff;
  --text: #27313a;
  --muted: #6c747b;
  --border: #ddd8cc;
  --danger: #a52a2a;
  --success: #2d6a4f;
  --shadow: 0 10px 28px rgba(0, 0, 0, 0.12);
```

}

5.2 CSS 3D transforms and transitions

The Rooms page uses perspective, preserve-3d, backface-visibility and rotateY(180deg) to create the flipcard effect. The current simplified CSS uses perspective: 1000px and a 0.7-second transform transition.

NEW CODE - Current CSS 3D flip card

```
flip-card { height:
390px; perspective:
1000px;
}

.flip-card-inner { position:
relative; width: 100%;
height: 100%; transition:
transform 0.7s ease; transform-
style: preserve-3d;
}

.flip-card:hover .flip-card-inner {
transform: rotateY(180deg); }
```

5.3 Grid, Flexbox and responsive design

CSS Grid and Flexbox are used for the header, facilities, room cards, booking layout, gallery and contact cards. The current stylesheet uses responsive breakpoints at 850px and 580px. Multi-column layouts reduce to fewer columns and then one column on small screens.

NEW CODE - Current responsive breakpoints

```
@media (max-width: 850px) {
   .welcome-section,
   .media-section,
   .booking-layout, .contact-
layout { grid-template-
columns: 1fr;
   }

   .facility-grid,
   .room-card-grid, .gallery-
grid {
      grid-template-columns: repeat(2, 1fr);
   }
}

@media (max-width: 580px) {
   .facility-grid,
   .room-card-grid,
```

```
    .gallery-grid,     .date-grid {
grid-template-columns:  1fr;
    }
}
```

5.4 Form validation states and UI feedback

JavaScript adds invalid or valid classes to form controls. CSS then gives invalid fields the danger border color and valid fields the success border color. Separate message classes provide readable inline feedback.

NEW CODE - Current validation feedback styles

```
input.invalid, select.invalid {      border-color: var(--
danger);
}

input.valid, select.valid {
border-color: var(--success);
}

.error-message {
display: block;     min-
height: 20px;
margin-top: 5px;
color: var(--danger);
font-size: 0.86rem; }
```

6. Requirement Traceability

The table below maps the Assignment 2 requirements .

Assignment 2 Requirement	Status / Evidence	Primary File(s)
Continue Assignment 1 website	Completed - five-page Royal Paradise hotel website retained and enhanced	All HTML pages
At least 5 new HTML elements/CSS styles	video/source, button, article, aside, fieldset/legend, small, dl/dt/dd, 3D transforms, transitions and responsive Grid/Flexbox	HTML + style.css
Use at least 3 listed examples	HTML5 video + CSS 3D transforms + CSS transitions	index.html, rooms.html, style.css
2(a) Timer-based content/style change	Offer text and CSS class change every 3 seconds using setInterval()	main.js
2(b) User-action DOM/event handling	Show/Hide Facilities using getElementById(), addEventListener(), classList.toggle() and textContent	main.js
2(c) Own input validity code	Password requires 8+ characters, uppercase, lowercase and a number using regular expressions	booking.js

2(d) Validate >=3 form elements and prevent invalid submit	Validates name, email, password, room and dates; uses event.preventDefault()	booking.js
2(e) Other sophisticated JavaScript	Live room estimate with room data, date arithmetic, checked facilities and DOM updates	booking.js
Assignment 2 Requirement	**Status / Evidence**	**Primary File(s)**
Website / webpage UI design	Shared navigation, luxury palette, cards, Grid/Flexbox, media queries and feedback states	style.css
Report screenshots + code highlights	Current screenshots of all five webpages.	This report

7. Screenshots

7.1 Home Page

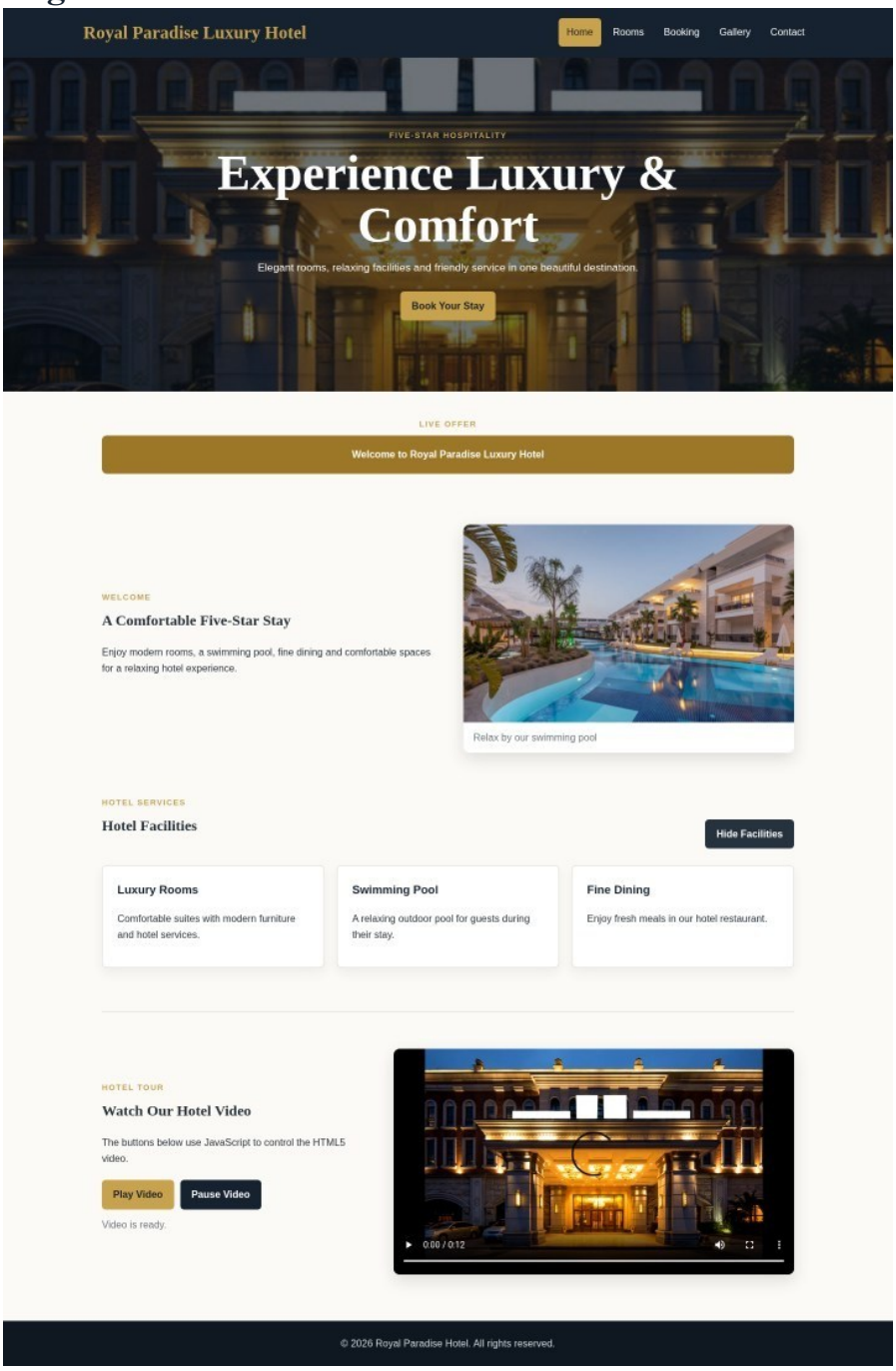


Figure 3: Current Home page with the facilities revealed; timed offer and HTML5 video section.

7.2 Rooms Page

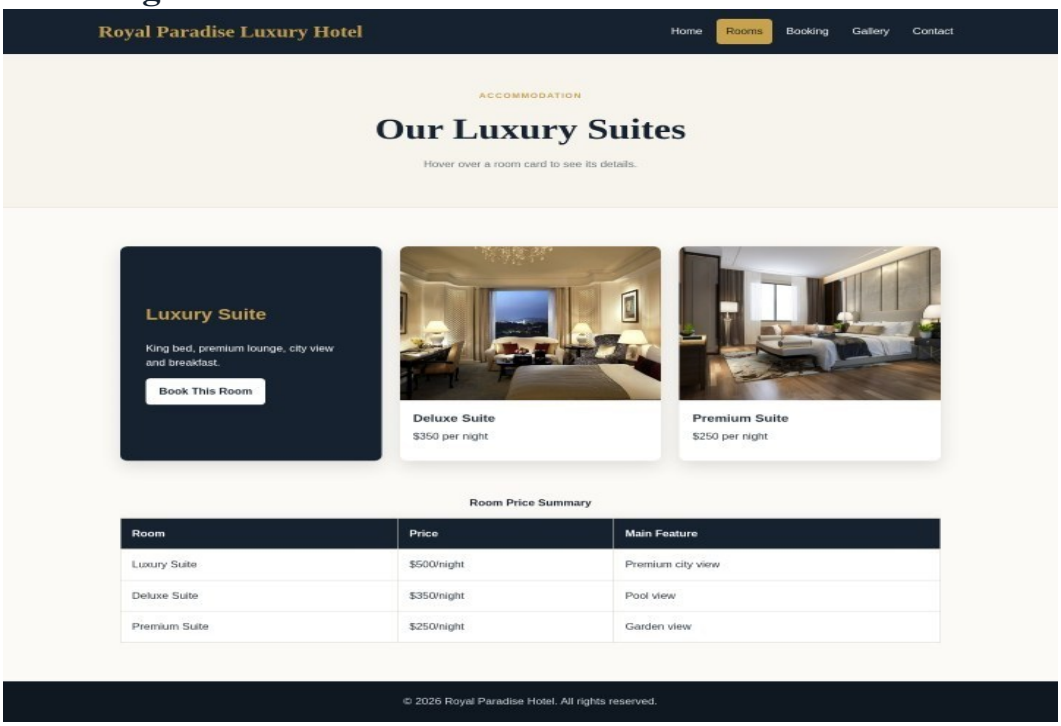


Figure 4: Current Rooms page; the first room card is shown in its hovered/flipped state.

7.3 Booking Page

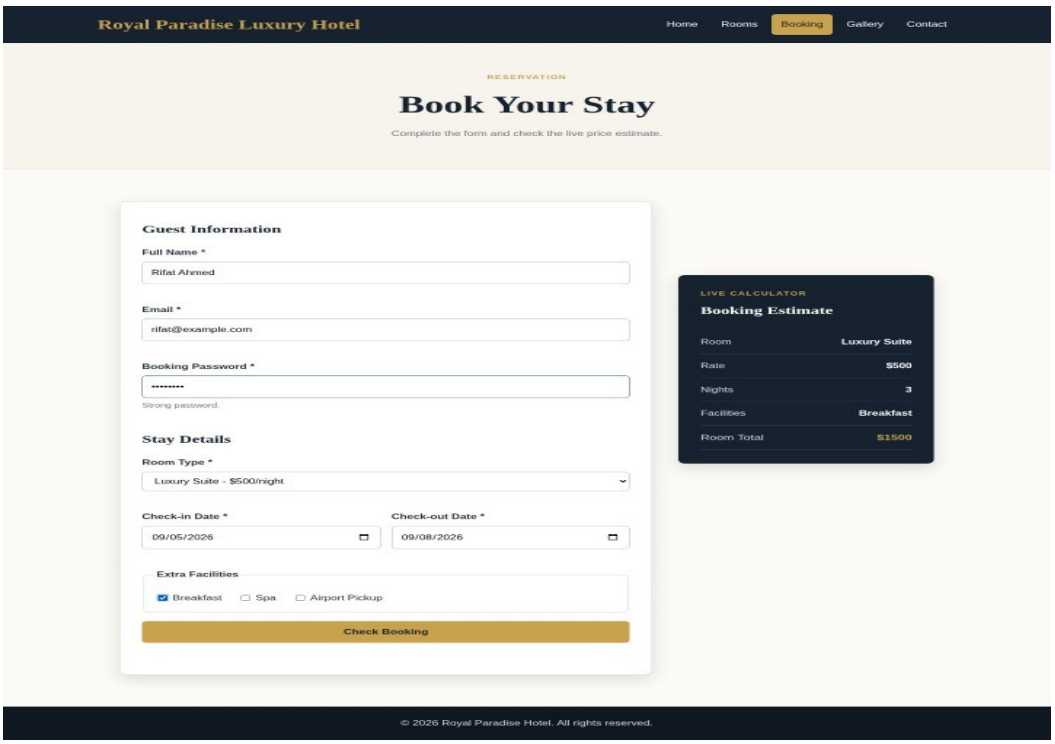


Figure 5: Current Booking page a live estimate of 3 nights at $500/night.

7.4 Gallery Page

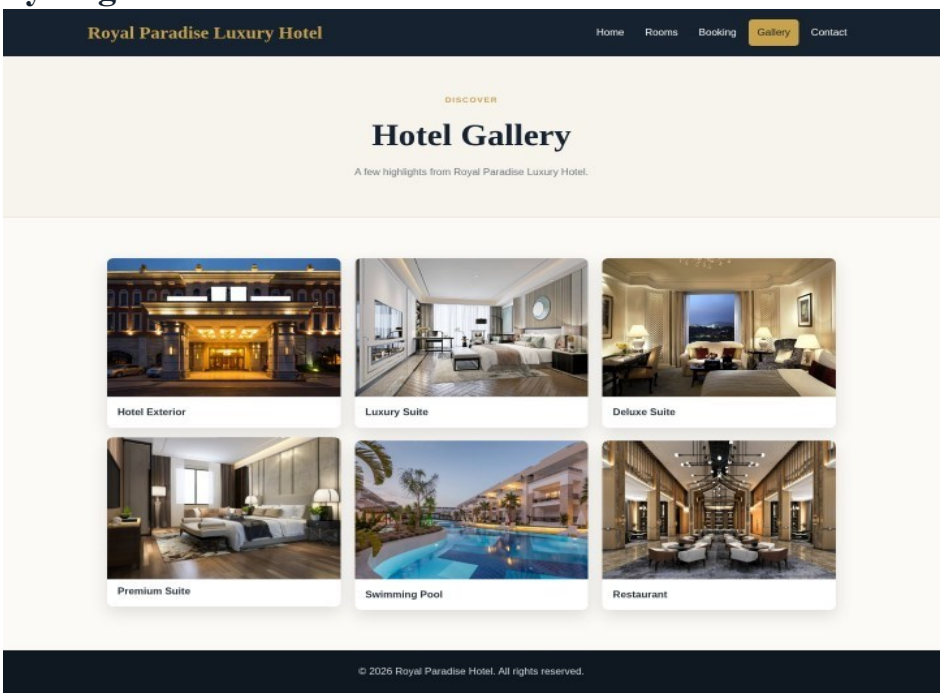


Figure 6: Current Gallery page showing the responsive six-image layout with CSS hover styling.

7.5 Contact Page

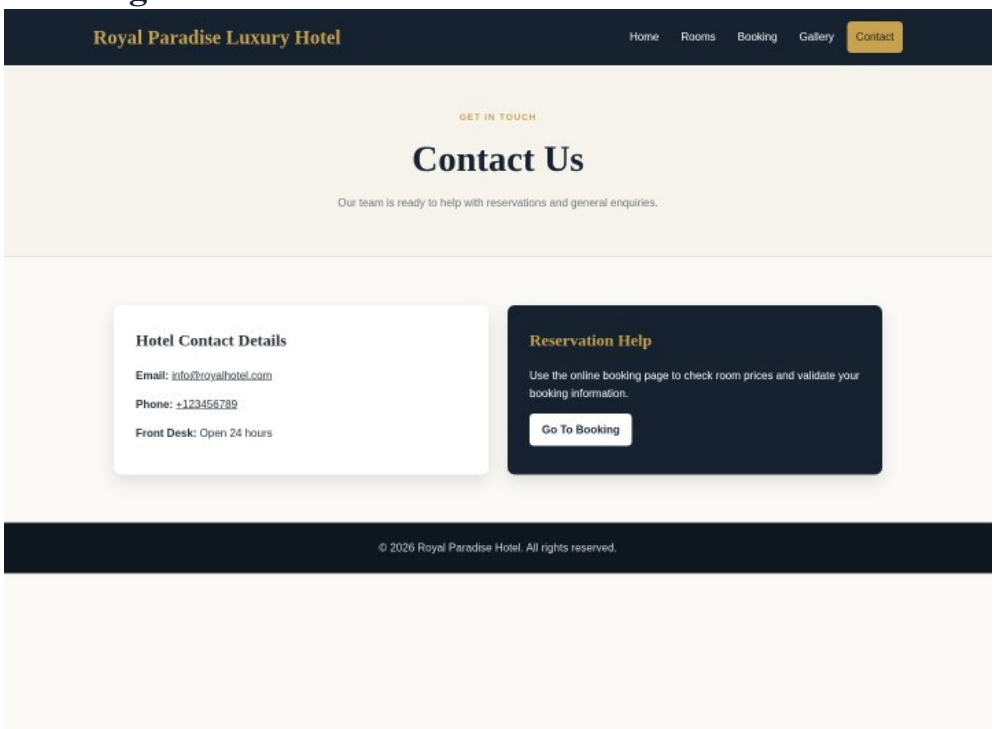


Figure 7: Current Contact page with contact details and the Booking call-to-action.

8. Conclusion

The Royal Paradise Luxury Hotel was successfully developed as a focused Assignment 2 front end website with HTML5 media, advanced CSS and JavaScript. The final project consists of five HTML

pages, a shared CSS file and two JavaScript files, which makes the implementation clear and easy to demonstrate.

The Home page has timed promotional content and Show/Hide Facilities interaction and JavaScript controls for the HTML5 video. CSS 3D flip cards and transitions are demonstrated in the Rooms page. The main JavaScript work is in the Booking page with custom password rules, validation of multiple form elements, prevention of invalid submission, date checking and a live room price calculator. The Gallery page has a responsive image grid with CSS hover effects, and the Contact page simply has contact and reservation links.